MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 4: DESIGNING A MOTION-ACTIVATED RFID SYSTEM

SECTION 1

GROUP 9

INSTRUCTOR: DR WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR ZULKIFLI BIN ZAINAL ABIDIN

NO	NAME	MATRIC NO
1	NOR EFFENDI BIN ANUAR @ ANWAR	2310519
2	NUR AINA SAFFIYA BINTI MOHD TARMIZI	2411644
3	NUR ATIKAH BINTI KAMARUL BAHARIN	2319846

DATE OF SUBMISSION: 1 APRIL 2026

TABLE OF CONTENTS

1.0 INTRODUCTION

In this experiment, MPU6050 sensor and RFID module are being introduced. The MPU6050 is a 6-axis motion tracking device, which has a 3-axis gyroscope and 3-axis accelerometer. Radio Frequency Identification or known as RFID is a form of wireless communication that uses electromagnetic in the radio frequency to identify an object or person.

For task 1, MPU6050 is used to detect circular motion to activate the LED. The circular motion will be detected by MPU6050 and sent the input to the Python which will send a command to Arduino to turn the LED on.

Furthermore, on Task 2, the RFID module is used to detect the RFID card when it is close to the RFID reader/writer. The RFID will send input signal to Python which will send command signal to Arduino to turn the LED on.

Finally on Task 3, it used the combination of RFID module MPU6050 sensor to simulate opening and closing the door. By doing the circular motion with MPU6050 and then scan the RFID card, the servo will move as in opening the door, the red LED will turn off and the green LED is turned on to show that the door is open.

The goal for this experiment is to understand the use of the MPU6050 sensor and RFID module during the experiment and relate it to real devices or machines that use both components and how it works.

2.0 MATERIALS AND EQUIPMENT

2.1 Task 1

- Microcontroller: Arduino Uno and USB cable
- Input component: MPU6050
- Output component: LED
- Resistor: 220Ω
- Prototyping: Breadboard and Jumper wires
- Software: Arduino IDE and Python (PyCharm)
- Libraries:

  - Arduino: MPU6050.h and Wire.h

  - Python: *pyserial* and *matplotlib*

2.2 Task 2

- Microcontroller: Arduino Uno and USB cable
- Input component: RFID module
- Output component: LED
- Resistor: 220Ω
- Prototyping: Breadboard, Jumper wires and RFID card
- Software: Arduino IDE and Python (PyCharm)
- Libraries:

  - Python: *pyserial* and *matplotlib*

2.3 Task 3

- Microcontroller: Arduino Uno and USB cable
- Input component: MPU6050 and RFID module

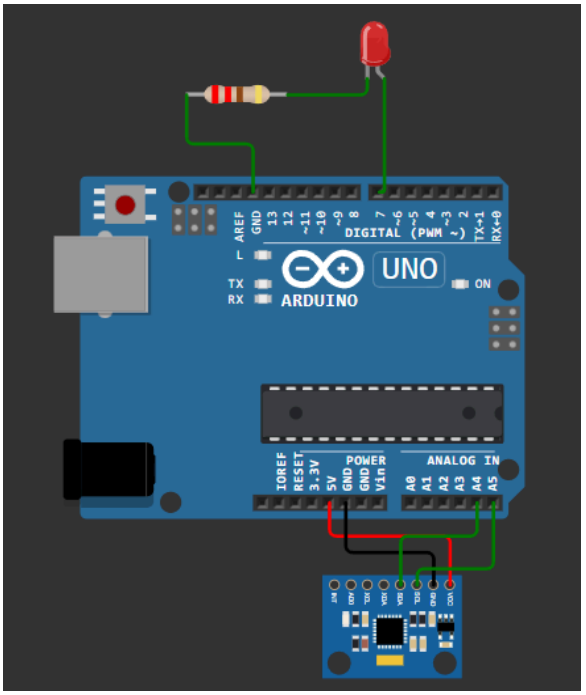
- Output component: LED Red, LED Green and Servo motor
- Resistor: 2x 220Ω
- Prototyping: Breadboard, Jumper wires and RFID card
- Software: Arduino IDE and Python (PyCharm)
- Libraries:

  - Arduino: Servo.h, MPU6050.h and Wire.h library

  - Python: *pyserial* and *matplotlib*

3.0 EXPERIMENTAL SETUP

3.1 Task 1



Picture 3.1 - Circuit assembly for Task 1

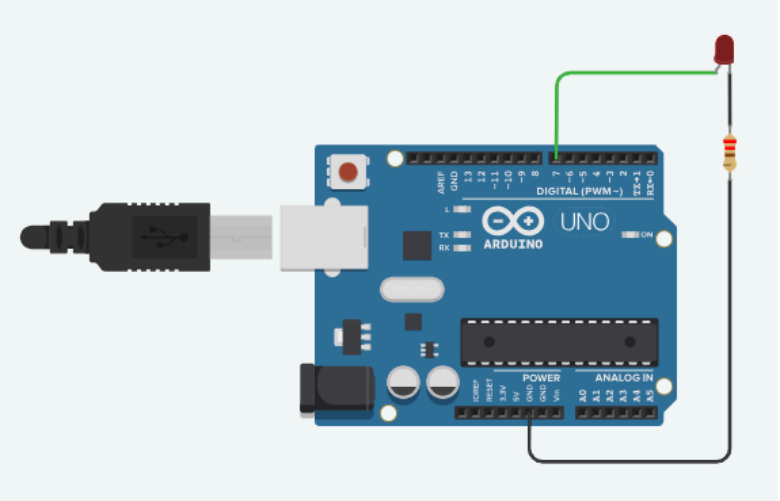
1. Connect the MPU6050 to the Arduino Uno:

- Connect the VCC pin on MPU6050 to the 5V pin on Arduino Uno.
- Connect the GND pin on MPU6050 to the GND pin on Arduino Uno.
- Connect the SDL pin on MPU6050 to the analog pin (A5).
- Connect the SDA pin on MPU6050 to the analog pin (A4).

2. Connect the LED to the Arduino:

- Connect the cathode or negative leg of the LED to the GND pin.
- Connect the 220Ω resistor to the anode or positive leg of the LED.
- Connect the other end of the resistor to one of the digital pins (D7).

3.2 Task 2



Picture 3.2 - Circuit assembly for Task 2 (for LED connection)

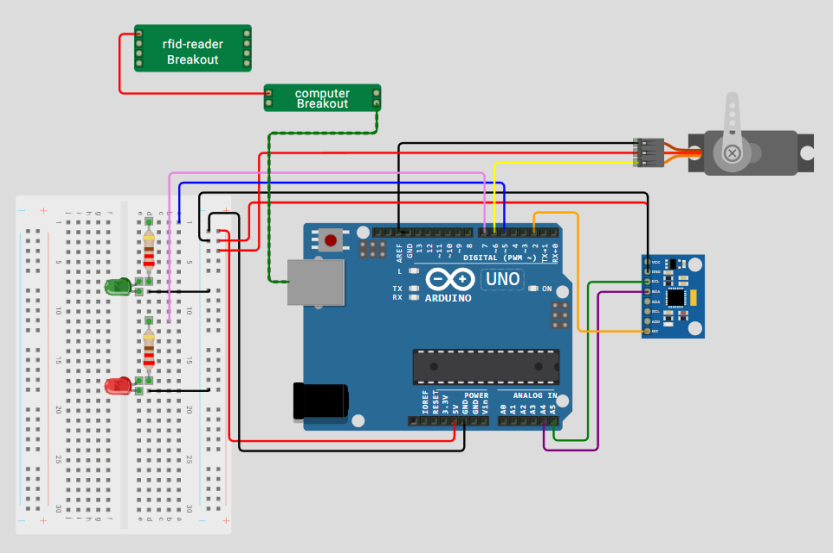
1. Connect the LED to the Arduino:

   - Connect the anode or positive leg of the LED to one of the digital pins (D7).
   - Connect the 220Ω resistor to the cathode or negative leg of the LED.
   - Connect the other end of the resistor to the GND pin.

2. RFID connection for Task 2:

   - Connect the USB part of the RFID Reader/Writer to the PC.
   - The Python will read the RFID card and send a signal to Arduino through PC connection.

3.3 Task 3



Picture 3.2 - Circuit assembly for Task 3

1. Connect the Red LED to the Arduino:

- Connect the cathode or negative leg of the LED to the GND pin.
- Connect the 220Ω resistor to the anode or positive leg of the LED.
- Connect the other end of the resistor to one of the digital pins (D7)

2. Connect the Green LED to the Arduino:

- Connect the cathode or negative leg of the LED to the GND pin.
- Connect the 220Ω resistor to the anode or positive leg of the LED.
- Connect the other end of the resistor to one of the digital pins (D5).

3. Connect the servo to the Arduino:

- Connect the brown wire to the GND pin.
- Connect the red wire to the 5V terminal on the breadboard.
- Connect the orange wire to one of the digital pins (D6).

4. Connect the MPU6050 to the Arduino Uno:

- Connect the VCC pin on MPU6050 to the 5V terminal on the breadboard.
- Connect the GND pin on MPU6050 to the GND terminal on the breadboard.
- Connect the SDL pin on MPU6050 to the analog pin (A5).
- Connect the SDA pin on MPU6050 to the analog pin (A4).

5. RFID connection for Task 3:

- Connect the USB part of the RFID Reader/Writer to the PC.
- The Python will read the RFID card and send a signal to Arduino through PC connection.

4.0 METHODOLOGY

4.1 Task 1

4.1.1 Circuit Setup

The hardware configuration focuses on interfacing the MPU6050 with the Arduino Uno to capture motion data.

- **MPU6050 Connection**: The sensor is connected with VCC to 5V, GND to GND, SDA to pin A4, and SCL to pin A5.
- **Indicator LED**: A single LED is connected to digital pin D7, with a 220 Ω current-limiting resistor connected to GND.
- **Stability**: Jumper wires and a breadboard are used to ensure secure connections for the data stream.

4.1.2 Program Development and Uploading

- **Arduino Sketch**: The *MPU6050.h* and *Wire.h* libraries are utilized to initialize the sensor. The code is written to capture raw accelerometer and gyroscope data and stream it through the Serial port at 9600 baud.
- **Python Script**: A Python script using *pyserial* is established to read the incoming COM port data.
- **Uploading**: The Arduino code is compiled and uploaded via the Arduino IDE, followed by closing the Serial Monitor to allow the Python script to claim the COM port.

4.1.3 Program Operation

- **Data Streaming**: The Arduino constantly sends formatted X-Y-Z accelerometer data to the computer.
- **Motion Analysis**: The Python script analyzes the *history_x* and *history_y* buffers to detect circular motion by calculating the center (mean), checking amplitude/variance, and verifying symmetry.
- **Feedback Loop**: Upon detecting a circle, Python sends a serial command ('O' for ON, 'F' for OFF) back to the Arduino to toggle the LED. A "debounce" clearing of the buffer is performed to prevent multiple triggers from one motion.

4.1.4 Coding

Arduino Code:

```
// --- DEFINE YOUR LED PIN HERE ---
const int ledPin = 7;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Start with LED OFF
}

void loop() {
  // Listen for commands from Python
  if (Serial.available() > 0) {
    char cmd = Serial.read();

    if (cmd == '1') {
      digitalWrite(ledPin, HIGH); // Turn LED ON
    }
    else if (cmd == '0') {
      digitalWrite(ledPin, LOW);  // Turn LED OFF
    }
  }
}
```

Python Code:

```
import serial
import numpy as np
import time
from collections import deque

# -- Replace 'COMx' with your Arduino's serial port
SERIAL_PORT = 'COM5'
BAUD_RATE = 9600

# Buffers to store the last 40 data points (approx. 2 seconds of
motion)
history_x = deque(maxlen=40)
history_y = deque(maxlen=40)


def is_circular_motion(x_data, y_data):
    if len(x_data) < 10:
        return False

    # Convert to numpy arrays
    x = np.array(x_data)
    y = np.array(y_data)

    # -------------------------
    # Step 1: Find center (mean)
    # -------------------------
    cx = np.mean(x)
    cy = np.mean(y)

    # Center the data
    x_centered = x - cx
    y_centered = y - cy

    # -------------------------
    # Step 2: Check amplitude (movement strength)
    # -------------------------
    variance = np.var(x_centered) + np.var(y_centered)

    if variance < 50:  # <-- tune this threshold
        return False

    # -------------------------
    # Step 3: Check symmetry (circle vs line)
    # -------------------------
    std_x = np.std(x_centered)
    std_y = np.std(y_centered)
```

```
    if std_y == 0:
        return False

    ratio = std_x / std_y

    if ratio < 0.5 or ratio > 2.0:
        return False

    # -------------------------
    # Step 4: Check rotation direction (cross product)
    # -------------------------
    cross_products = []

    for i in range(len(x_centered) - 1):
        x1, y1 = x_centered[i], y_centered[i]
        x2, y2 = x_centered[i + 1], y_centered[i + 1]

        cross = x1 * y2 - y1 * x2
        cross_products.append(cross)

    # -------------------------
    # Step 5: Count direction consistency
    # -------------------------
    positive = sum(1 for c in cross_products if c > 0)
    negative = sum(1 for c in cross_products if c < 0)

    total = len(cross_products)

    if total == 0:
        return False

    # Check if mostly one direction (>70%)
    if positive / total > 0.7 or negative / total > 0.7:
        return True

    return False


def main():
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)  # Wait for Arduino to reset
     print("System ready. Move the MPU6050 in a circle to toggle the
LED...")

    # Keep track of the LED state
    led_is_on = False
```

```
        try:
            while True:
                raw = ser.readline().decode(errors='ignore').strip()
                if raw:
                    parts = raw.split(',')

                    if len(parts) >= 3:
                        try:
                            ax = int(parts[0])
                            ay = int(parts[1])

                            # Add new data to our buffers
                            history_x.append(ax)
                            history_y.append(ay)

                                # Only analyze if our buffer is full (40
points)
                            if len(history_x) == 40:
                                        if is_circular_motion(history_x,
history_y):

                                    # Toggle the state
                                    led_is_on = not led_is_on

                                    if led_is_on:
                                         print("Circle Detected! Toggling
LED: ON")
                                        ser.write(b'O')
                                    else:
                                         print("Circle Detected! Toggling
LED: OFF")
                                        ser.write(b'F')

                                    # DEBOUNCE: Clear the buffers!
                                        # Forces the system to wait for 2
full seconds of
                                         # NEW data before it can trigger
another toggle.
                                    history_x.clear()
                                    history_y.clear()

                            except ValueError:
                                pass  # Ignore corrupted serial lines

        except KeyboardInterrupt:
            print("\nProgram terminated by user.")
        finally:
            ser.write(b'F')  # Ensure LED turns off when script stops
```

```
        ser.close()
        print("Serial connection closed.")


if __name__ == "__main__":
    main()
```

4.2 Task 2

4.2.1 Circuit Setup

- **RFID Reader**: A USB RFID scanner is connected directly to the computer's USB port. It functions as a Human Interface Device (HID), essentially acting as a keyboard.
- **Output Hardware**: The LED remains on the breadboard, with the Anode connected to a designated digital pin and the Cathode to GND via a 220 Ω resistor.

4.2.2 Program Development and Uploading

- **Arduino Sketch**: The code is simplified to act as a listener. It monitors the Serial buffer for specific characters: '1' to set the *ledPin* HIGH and '0' to set it LOW.
- **Python Script**: The script uses the *input()* function to capture the 10-digit UID typed by the RFID scanner. An *AUTHORIZED_CARD* variable is defined with the specific UID found at the back of the RFID card provided.

4.2.3 Program Operation

- **Authentication**: When a card is scanned, the Python script compares the input UID against the authorized list.

- **Command Execution**: If the UID matches, Python sends a '1' to the Arduino, waits for 3 seconds, and then sends a '0' to turn the LED off.
- **Error Handling**: If an unauthorized card is scanned, the system prints a denial message and keeps the LED OFF.

4.2.4 Coding

Arduino Code:

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo gateServo;

const int greenLedPin = 5;
const int servoPin = 6;
const int redLedPin = 7;

const unsigned long streamDurationMs = 3000; // Stream data for 2
seconds
const unsigned long sampleDelayMs = 100;      // 20 Hz sample rate

void setup() {
  pinMode(greenLedPin, OUTPUT);
  pinMode(redLedPin, OUTPUT);

  gateServo.attach(servoPin);
  gateServo.write(90); // Start at closed position

  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  // Default state: red LED ON (idle/unauthorized)
  digitalWrite(redLedPin, HIGH);
```

```
  digitalWrite(greenLedPin, LOW);
}

void openGate() {
  for (int pos = 90; pos <= 180; pos += 2) {
    gateServo.write(pos);
    delay(15);
  }
  Serial.println("Servo opened (90 -> 180)");
}

void closeGate() {
  for (int pos = 180; pos >= 90; pos -= 2) {
    gateServo.write(pos);
    delay(15);
  }
  Serial.println("Servo closed (180 -> 90)");
}

void wrongCardFlash() {
  digitalWrite(redLedPin, HIGH);
  digitalWrite(greenLedPin, LOW);
  delay(100);
  // Blink red LED 3 times
  for (int i = 0; i < 3; i++) {
    digitalWrite(redLedPin, HIGH);
    delay(150);
    digitalWrite(redLedPin, LOW);
    delay(150);
  }
  digitalWrite(redLedPin, HIGH); // Return to idle red ON
}

void loop() {
  if (Serial.available()) {
    char cmd = Serial.read();

    if (cmd == '1') {
      // Access granted
      digitalWrite(greenLedPin, HIGH);
      digitalWrite(redLedPin, LOW);
      openGate();
    }
    else if (cmd == '0') {
      // System disarmed (Lock)
      digitalWrite(greenLedPin, LOW);
      digitalWrite(redLedPin, HIGH);
      closeGate();
```

```
    }
    else if (cmd == 'X') {
      // Wrong card feedback
      wrongCardFlash();
    }
    else if (cmd == 'R') {
      // Python requested motion data
      unsigned long start = millis();
      while (millis() - start < streamDurationMs) {
        int16_t ax, ay, az, gx, gy, gz;
        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

        Serial.print(ax); Serial.print(", ");
        Serial.print(ay); Serial.print(", ");
        Serial.print(az); Serial.print(", ");
        Serial.print(gx); Serial.print(", ");
        Serial.print(gy); Serial.print(", ");
        Serial.println(gz);

        delay(sampleDelayMs);
      }
    }
  }
}
```

Python Code:

```
import serial
import time

# --- CONFIGURATION ---
SERIAL_PORT = 'COM5'
BAUD_RATE = 9600

# Fill in your card's unique 10-digit ID number here
AUTHORIZED_CARD = ("0003855091")

def main():
    # Establish connection with the Arduino
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)  # Wait for Arduino to reset

    print("RFID System Ready.")
    print("Scan a card to test the system. Press CTRL+C to stop.")

    try:
        while True:
```

```
                    # The USB scanner types the UID and hits Enter
automatically.
            # input() pauses the script and waits for this to happen.
            uid = input("\nTap card: ").strip()

            if uid == AUTHORIZED_CARD:
                      print(f"[{uid}] - Card Authorized! Turning LED
ON...")
                arduino.write(b'1')  # Send ON command to Arduino

                time.sleep(3)  # Keep LED on for 3 seconds

                arduino.write(b'0')  # Send OFF command
                print("LED OFF. Ready for next scan.")

            else:
                    print(f"[{uid}] - Unauthorized Card! LED remains
OFF.")

    except KeyboardInterrupt:
        print("\nExiting program.")
    finally:
        # Ensure the LED is safely turned off when the program closes
        arduino.write(b'0')
        arduino.close()
        print("Serial connection closed.")

if __name__ == "__main__":
    main()
```

4.3 Task 3

4.3.1 Circuit Setup

- **Servo Motor**: An SG90 servo is added, with the Signal wire connected to D6, VCC to 5V, and GND to GND.
- **Dual LED Feedback**: A Green LED is connected to D5 (Authorized) and a Red LED to D7 (Unauthorized/Idle).
- **IMU Integration**: The MPU6050 remains connected with pins A4/A5 as established in Task 1.

4.3.2 Program Development and Uploading

- **Integrated Arduino Sketch**: The code includes *Servo.h* to control the gate position (90° for closed, 180° for open). It adds a case 'R' to trigger a 2-second burst of MPU6050 data streaming only when requested.

4.3.3 Program Operation

- Two-Factor Sequence:
    1. **RFID Phase**: The user taps their card; Python verifies the UID
    2. **Motion Phase**: If the card is valid, the system prompts the user to perform a circular motion. Python sends 'R' to the Arduino to begin sampling.
    3. **Verification**: Python calculates the *net_rotation*. If the rotation exceeds 100° or shows a wide range, access is granted.
- Final Action: On success, Arduino receives '1', turns on the Green LED, and moves the servo to 180°. On failure, the Red LED flashes through the *wrongCardFlash()* function.

4.3.4 Coding

Arduino code:

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

// --- DEFINE YOUR LED PIN HERE ---
const int ledPin = 7; // Fill in

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();
```

```
  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Start with LED off
}

void loop() {
  // 1. Read motion data
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  // 2. Send formatted data to Python (ax,ay,az)
  Serial.print(ax); Serial.print(',');
  Serial.print(ay); Serial.print(',');
  Serial.println(az);

  // 3. Listen for commands from Python to control the LED
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'O') {
      digitalWrite(ledPin, HIGH); // Turn LED ON
    }
    else if (cmd == 'F') {
      digitalWrite(ledPin, LOW);  // Turn LED OFF
    }
  }

  delay(50); // Small delay for stability
}
```

Python code:

```
import serial
import time
import math

# --- CONFIGURATION ---
SERIAL_PORT = "COM5"  # Update with your Arduino port
BAUD_RATE = 9600
AUTHORIZED_ID = "0003855091"

# ---------------------

def detect_circle(rduino11):
```

```
    """ Asks Arduino to stream a burst of MPU data, calculates the
    rotation angles,and decides whether a circular motion happened."""
    print("\n[!] Requesting samples from Arduino… Move in a circle NOW!")
        rduino11.reset_input_buffer()
        rduino11.write(b'R')  # Trigger Arduino data stream

        points = []
        start_time = time.time()
        timeout = 3.0

    # Read until timeout (Arduino stops streaming automatically after 2s)
        while time.time() - start_time < timeout:
            raw = rduino11.readline().decode(errors='ignore').strip()
            if not raw:
                continue

            parts = raw.split(', ')
            if len(parts) >= 6:
                try:
                    ax = int(parts[0])
                    ay = int(parts[1])
                    az = int(parts[2])
                    points.append((ax, ay, az))
                except ValueError:
                    continue

        print(f"Collected {len(points)} samples.")

        if len(points) > 0:
            print(f"DEBUG - First Point: {points[0]}")
            print(f"DEBUG - Last Point: {points[-1]}")
        if len(points) < 8:
            print("Not enough motion data - try moving more
continuously.")
            return False

    # Build angles from (ax, ay) and unwrap to follow continuous rotation
        angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]

        unwrapped = [angles[0]]
        for a in angles[1:]:
            prev = unwrapped[-1]
            diff = a - prev

    # Unwrap angles to avoid -180/180 discontinuity leaps
            if diff > 180:
                diff -= 360
            elif diff < -180:
```

```
                diff += 360
            unwrapped.append(prev + diff)

    # Compute net rotation and range
        net_rotation = unwrapped[-1] - unwrapped[0]
        rotation_range = max(unwrapped) - min(unwrapped)

        print(f"Net rotation:{net_rotation: .1f}°, Range:
{rotation_range: .1f}°")

    # Lenient detection logic (either a decent net rotation OR wide
range)
        if abs(net_rotation) > 60 or rotation_range > 80:
            print(" Circle gesture detected!")
            return True

    # Extra check: count direction-consistent large angle deltas
        deltas = [unwrapped[i + 1] - unwrapped[i]
        for i in range(len(unwrapped) - 1)]
        large_moves = sum(1 for d in deltas if abs(d) > 5)

        if large_moves >= max(6, len(points) // 4):
            print(" Motion had multiple consistent movements - accepting
as circle.")
            return True

        print(" Motion insufficient.Try a clearer circular motion.")
        return False


    def main():
        rduino12 = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)  # Wait for Arduino to reset
        print("System Ready. Waiting for RFID…")

        armed = False  # System starts locked

        try:
            while True:
                # USB RFID acts as a keyboard
                card = input("\nTap card: ").strip()

                if card == AUTHORIZED_ID:
                    armed = not armed  # Toggle state

                    if armed:
                      print("Card Accepted - Please perform circular
motion verification!")
```

```
                        if detect_circle(rduino12):
                            print("Gesture OK -> System UNLOCKED (LED
Green)")
                            rduino12.write(b'1')
                        else:
                            print("Wrong gesture -> Try again!")
                            armed = False  # Revert state if gesture
fails
                    else:
                        print("System Disarmed -> System LOCKED (LED
Red)")
                        rduino12.write(b'0')

                else:
                    print("WRONG CARD -> Access Denied")
                    rduino12.write(b'X')

        except KeyboardInterrupt:
            print("\nExiting.")

        finally:
                rduino12.write(b'0')
                rduino12.close()

    if __name__ == "__main__":
        main()
```

5.0 DATA COLLECTION

5.1 Task 1

The data collection process begins with the Arduino Uno capturing raw 16-bit signed integers from the MPU6050 sensor. These values represent linear acceleration and angular velocity across three axes, though the primary focus for this task is the *Ax* and *Ay* accelerometer data. The Arduino is programmed to stream these values to the computer via a serial connection at a baud rate of 9600. To ensure signal stability, a small delay of 50ms is implemented in the loop, resulting in a consistent sampling rate of approximately 20 Hz. The resulting output is a continuous stream of comma-separated values *(Ax, Ay, Az)* that can be viewed in real-time through the Serial Monitor or captured by a Python script.

5.2 Task 2

In this stage, data collection shifts from a continuous stream to an event-driven model using a USB RFID scanner. The scanner functions as a Human Interface Device (HID), thus it automatically types the 10-digit Unique Identification (UID) of a scanned tag followed by a carriage return. The Python script uses a standard *input()* function to pause execution and wait for this data to be entered into the terminal. Prior to the main execution, users are required to scan their card into a text editor to identify and record the specific 10-digit authorized number for the configuration.

5.3 Task 3

Data collection for the final integrated system is a multi-step process that combines the previous methods into a two-factor authentication sequence. The process is

triggered only after a successful RFID scan, at which point the Python controller sends an 'R' command to the Arduino. This triggers a specific 2-second burst of 6-axis data *(Ax, Ay, Az, Gx, Gy, Gz)* from the MPU6050. The Arduino uses a *while* loop to stream this high-resolution data at a 20 Hz sample rate until the two-second duration has elapsed, providing a complete packet of movement data for the gesture verification phase.

6.0 DATA ANALYSIS

6.1 Task 1

The Python script utilizes a *deque* buffer to maintain a rolling window of the most recent 40 data points, which captures roughly two seconds of physical motion. The analysis logic first determines the mean of the X and Y movement to establish a center point for the motion. It then calculates the variance of the data to ensure the movement is significant enough to ignore minor electronic jitters. Furthermore, the script assesses the symmetry of the motion by comparing the standard deviation of both axes to verify that the path is round rather than a straight line. Finally, it tallies clockwise versus counter-clockwise turns using a 2D cross-product; if the movement consistently turns in one direction (exceeding a 70% threshold), a circular motion is confirmed and the LED state is toggled.

6.2 Task 2

The data analysis for the RFID system is based on string matching and conditional logic. Once a UID is captured by the Python script, it is stripped of any whitespace and compared directly against the predefined *AUTHORIZED_CARD* variable. If the scanned UID matches the authorized ID, the system executes a "Card Authorized" routine, sending a '1' command to the Arduino to trigger the LED for three seconds. Conversely, if the ID does not match, the system identifies it as an "Unauthorized Card," prints a denial message, and ensures the LED remains in the OFF state by sending a '0' command.

6.3 Task 3

The analysis of the motion data in this task employs advanced trigonometric calculations to verify the gesture requirement. Python converts the *(Ax, Ay)* coordinates into angles using the *math.atan2* function and unwraps these angles to prevent discontinuities when the sensor passes the 180°/-180° threshold. The script then calculates the net rotation and the total rotation range; a gesture is accepted if the net rotation exceeds 100° or the range exceeds 140°. Additionally, the script checks for direction-consistent large moves to ensure the motion was a deliberate circle. If both the RFID and motion analysis return positive results, the Arduino is instructed to open the servo gate and illuminate the green LED.

7.0 RESULTS

7.1 Task 1

In this experiment, the arduino successfully received continuous data of X, Y and Z from the MPU 6050 and sent it to the python at 9600 baud rate. The X, Y and Z data is used to calculate the circular motion using matplotlib in the Pycharm. When the circular motion is detected, Pycharm will send data like ‘O’ for On, ‘F’ for Off back to the Arduino. Then the LED that connected with the Arduino will turn On. If the circular motion is detected again, then the LED will turn off back. The cycle is repeated.

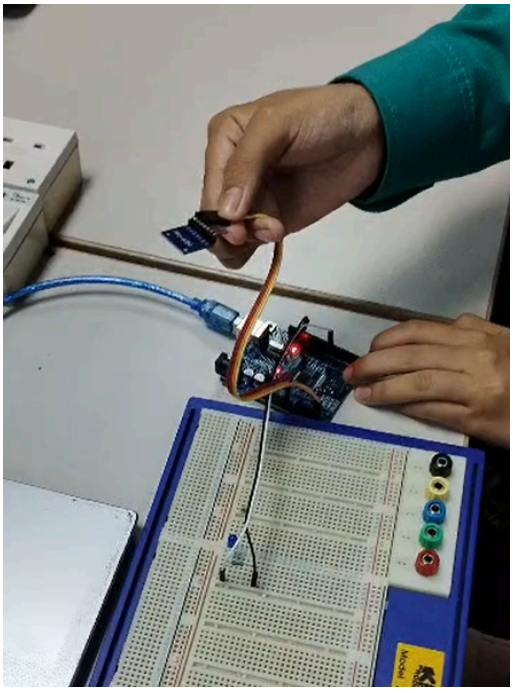


Diagram 7.1.1 show the MPU 6050 is move in circular motion

7.2 Task 2

In experiment 2, the serial number of the card is displayed at the terminal in the Pycharm when the card is scanned. If the serial number is the same as the number in the database, then the terminal in pycharm will display “Card authorize” and the LED connected to Arduino will turn On. The system has a delay of 3 seconds before the LED is turned Off back and the card is ready to scan back. After that, If the serial number of the card is not found in the database then at the terminal in Pycharm will display “Unauthorized card” and need to scan again.

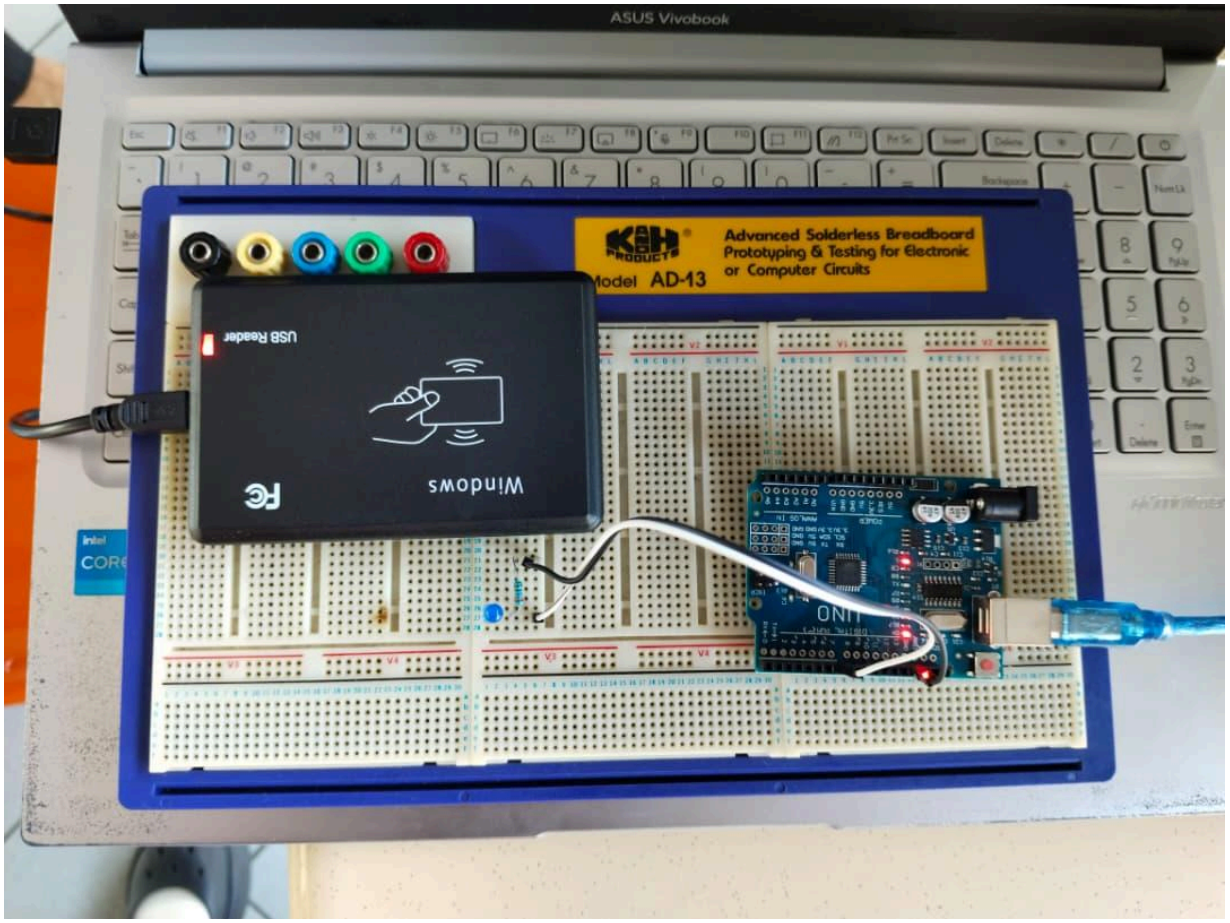


Diagram 7.2.1 show the connection of the RFID and the LED

7.3 Task 3

This is the combination of the 2 previous experiments. The integration between the circular motion of MPU 6050, RFID card system and movement of servo motors. The system starts with RFID card scanning. If the card's serial number is in the database, the system will proceed with the movement of the MPU 6050. The data of X, Y and Z from the MPU 6050 is collected by the Arduino Uno and sent to Pycharm with the baudrate of 9600. The Pycharm will do the math to calculate the circular motion of MPU 6050. If the motion is satisfied then Pycharm will send data to the Arduino to move the servo motor to 90 degrees and turn on the LED. The servo motor will keep at 90 degrees and the LED will keep on. When the card is scanned again then the servo motor will set to 0 degree and the LED will turn off. This is because the system is set as toggle action. The new action needs to be done to make the servo motor move 90 degrees or 0 degrees.

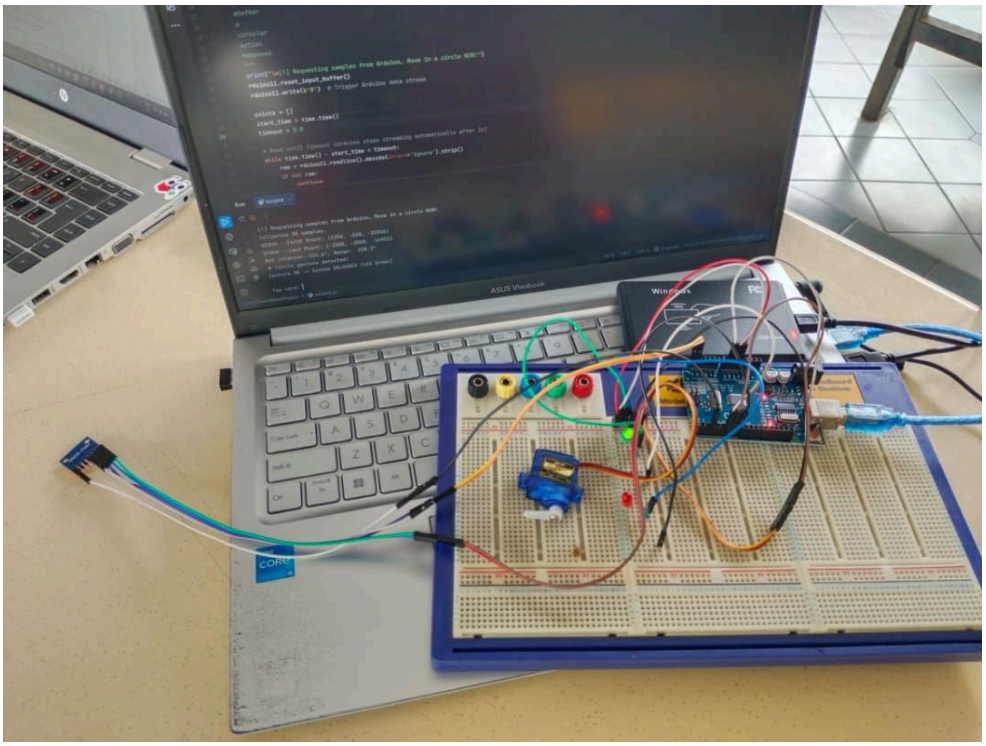


Diagram 7.3.1 shows the circuit connection of the system.

8.0 DISCUSSION

8.1 Task 1

The challenge for this experiment is to calculate the circular motion. We use matplotlib to help with the calculation. After that, we need to determine where the X coordinate is faced in the MPU 6050. Same goes with Y and Z coordinates. Then we know how the MPU 6050 should be positioned and then we can rotate the MPU 6050. There is also a delay from the LED. When the Pycharm detected the motion and sent the data back to the Arduino, it took time for the LED to turn On or Off.

8.2 Task 2

The connection for this experiment is much more simpler than the previous experiment because the RFID scanner is connected directly with the computer and the component connected with the Arduino Uno is led. After that, the most critical part in this experiment is that the serial number of the RFID card in the database must be the same as the registered card. Otherwise the system will not function properly even though we use the correct card.

8.3 Task 3

The crucial part in this experiment is the movement of the MPU 6050. The data must be collected correctly from the arduino before sent to the Pycharm to be calculated. The data must be checked at the serial monitor and make sure that all the Axis can be read. Missing space and the comma will make a single long number. Thus the Pycharm will not detect any motion from the MPU 6050. The connection also must be correct because this experiment consists of a lot of components and devices. One wrong connection can cause error and the system will not function well.

9.0 CONCLUSION

In conclusion, the lab successfully demonstrates the use of heavy mathematical processing and calculation. The experiment also shows how the arduino and pycharm communicate with each other to make an integration system consisting of calculation and hardware control. The RFID card can be integrated with a lot of other sensors to become a more functioning security system.

10.0 RECOMMENDATIONS

The recommendation for this lab is to make sure the serial monitor is closed before running the python script as only one process can occupy the COM port at a time. After that, the user must make sure to click the terminal in Pycharm before scanning because the scanner will type the data of the card wherever the cursor is currently active. Lastly, we can use the Kalman filter on the raw MPU 6050 data to reduce the noise before passing it.

11.0 REFERENCES

https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/

What is RFID? How It Works? Interface RC522 RFID Module with Arduino

https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

Security Access using MFRC522 RFID Reader with Arduino

https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/

Arduino Time Attendance System with RFID

https://www.instructables.com/Arduino-MFRC522-RFID-READER/

Arduino + MFRC522 RFID READER

ACKNOWLEDGEMENT

We would like to sincerely express our gratitudes to our instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal and the lab technician, for the help given during the task. The knowledge and suggestions are able to guide us throughout the progress of the task. We also would like to thank our classmates for their support and help which really give positive impacts to the progression of our task.

STUDENTS' DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed

unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature:  Read [/]

Name: NOR EFFENDI BIN ANUAR @ ANWAR Understand [/]

Matric Number: 2310519 Agree [/]

Signature:  Read [/]

Name: NUR AINA SAFFIYA BINTI MOHD TARMIZI Understand [/]

Matric Number: 2411644 Agree [/]

Signature:  Read [/]

Name: NUR ATIKAH BINTI KAMARUL BAHARIN Understand [/]

Matric Number: 2319846 Agree [/]